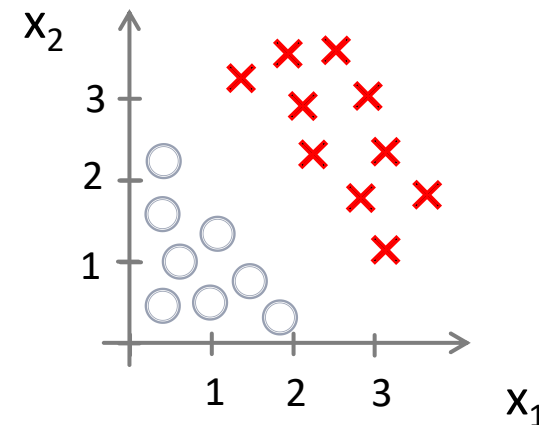
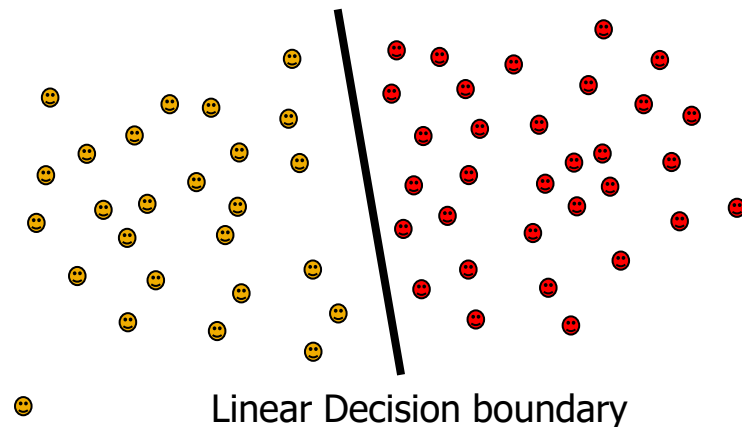


Linearly separable data

- The perceptron is a linear classifier
 - i.e., boundary can be specified by hyperplane
 - Example boundary: $w_0 + w_1x_1 + \dots + w_kx_k = 0$
 - (Predicts True if on one side of the hyperplane and False otherwise)
- Example: what formula defines the separating hyperplane for these data?

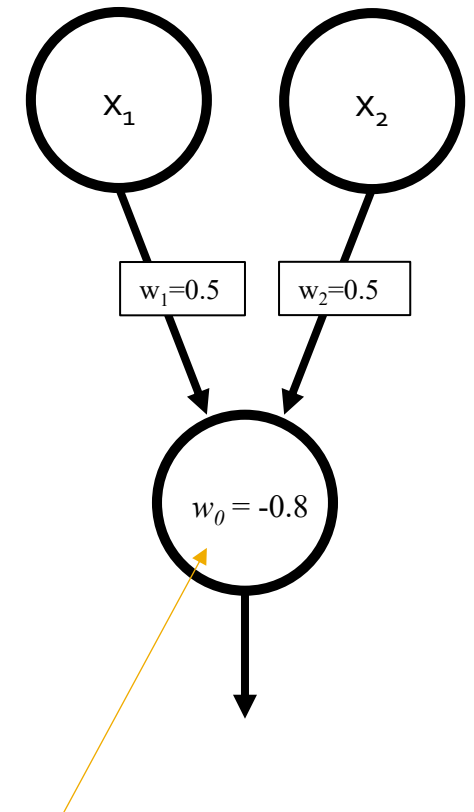


Perceptron: Examples

- A perceptron for AND:

x_1	x_2	y
1	1	T
1	0	F
0	1	F
0	0	F

- Two weights and intercept:
 - $h(x_i) = \text{Sign}(w_0 + w_1x_{i1} + w_2x_{i2})$
- One solution:
 - $w_1=0.5, w_2=0.5, w_0=-0.8$
- *Can you find another one?*

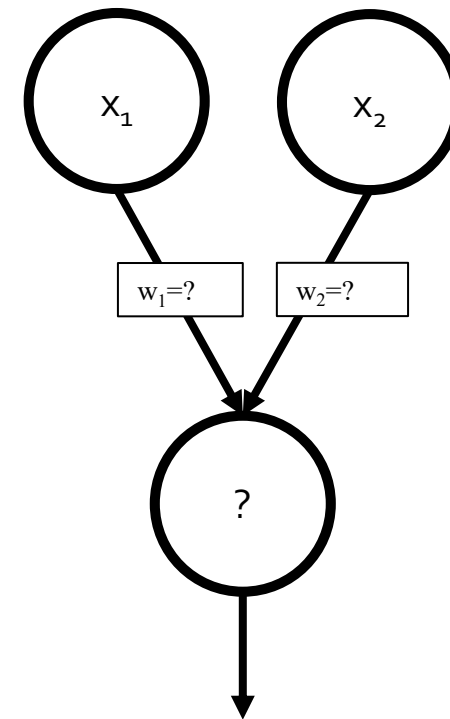


Note: You'll sometimes see a threshold T used instead of the bias w_0 , such that $T = -w_0$ (in this example, $T=0.8$)

Perceptron: Your turn

- A perceptron for OR:
 - Two weights and intercept:
 - $h(x_i) = \text{Sign}(w_0 + w_1x_{i1} + w_2x_{i2})$
 - Find possible weights w_0, w_1, w_2

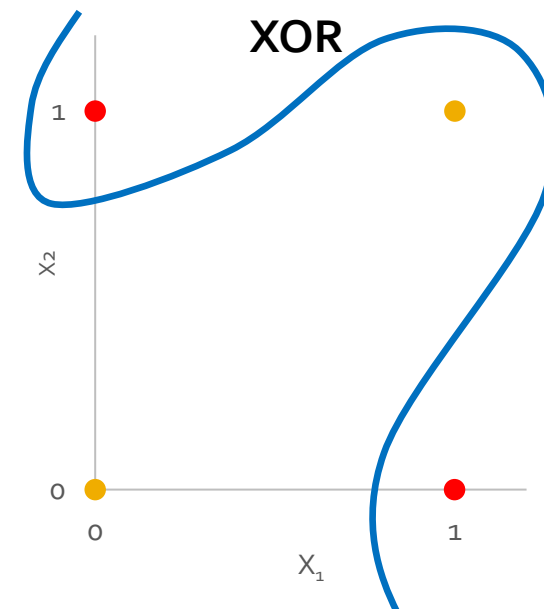
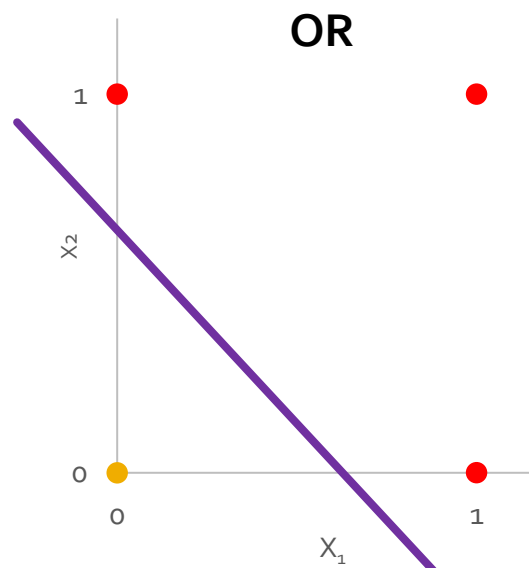
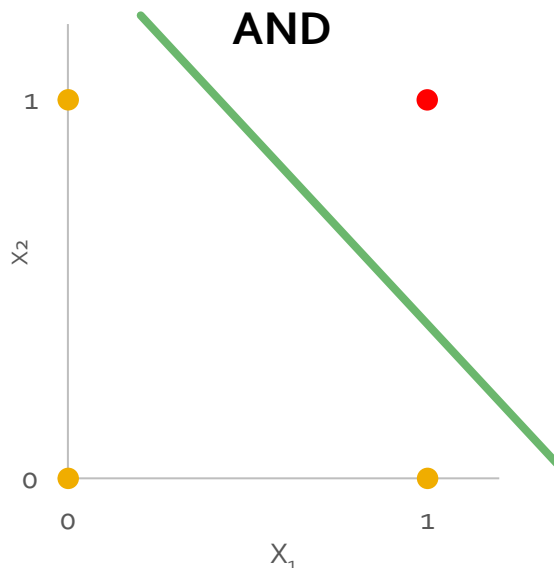
x_1	x_2	y
1	1	T
1	0	T
0	1	T
0	0	F



Perceptron: Examples

- You've seen AND and OR
- A perceptron for XOR?
 - Impossible! (Minsky & Papert 1969) → Why?
 - XOR is not **linearly separable**

x_1	x_2	y
1	1	F
1	0	T
0	1	T
0	0	F



Outline

- Neural Networks: Motivation and Biology
- The Perceptron
- **Learning perceptron weights**
- Multilayer networks

Learning weights

- Given we have input and output (for instance, a truth table), how do we learn the weights?
- Rosenblatt's **algorithm** provides a simple, error-driven update rule
 - Dates to 1957!

Learning weights (Rosenblatt)

- Rosenblatt's Algorithm (perceptron):

initialize weights randomly

while termination condition is not met:

 initialize $\Delta w_j = 0$

 for each training example (X_i, Y_i) :

 compute predicted output $\hat{Y}_i$

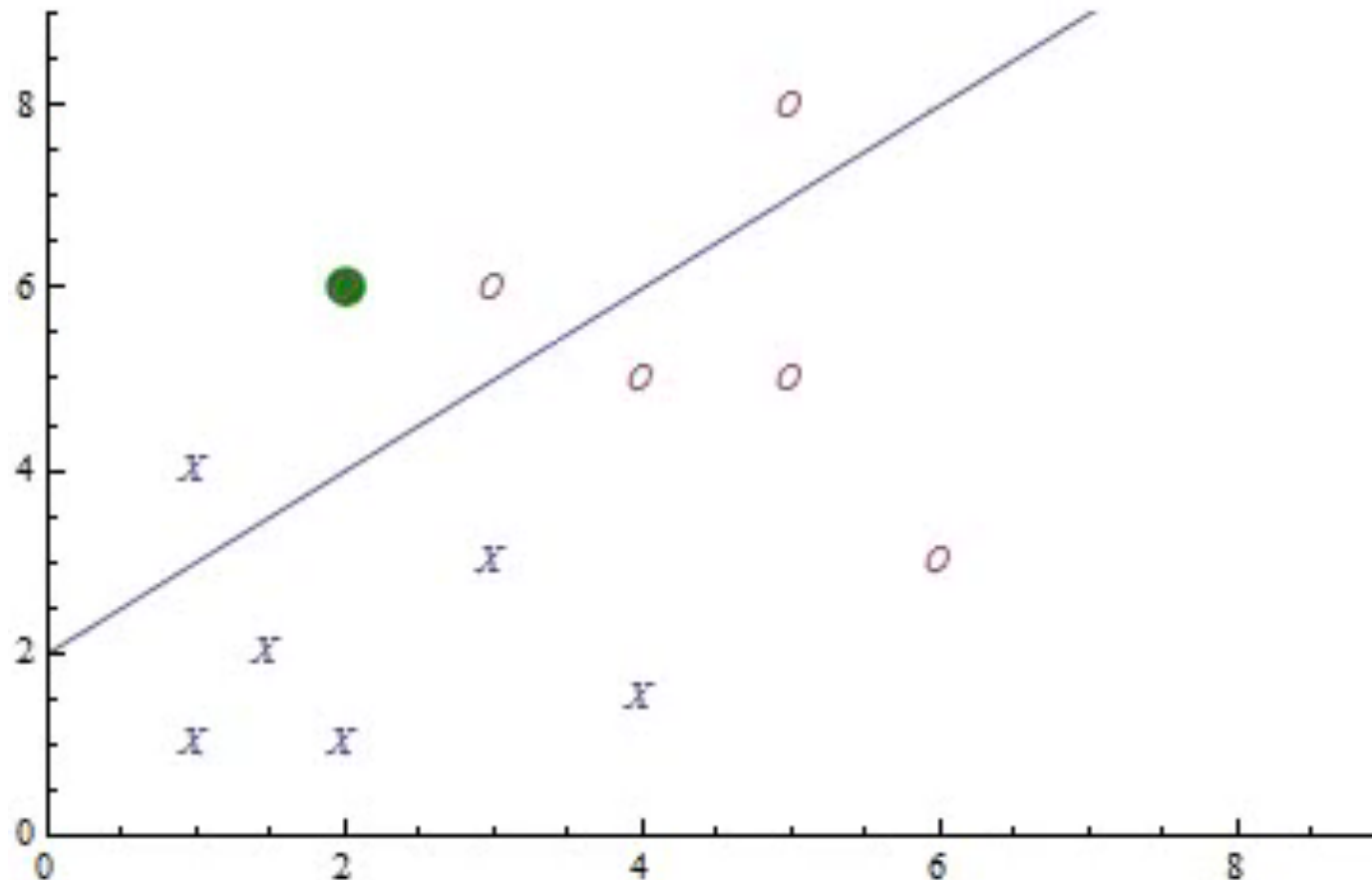
 foreach weight w_j :

$w_j = w_j + \eta (Y_i - \hat{Y}_i) X_{ij}$ ← "error-driven" learning

← Learning rate

Perceptron: In action

- Each misclassified point triggers an update that shifts/rotates boundary



Who cares?

- Rosenblatt proved the algorithm is guaranteed to converge as long as:
 - Training data are linearly separable
 - Learning rate is sufficiently small
 - (In the proof, it has to be infinitesimally small)

Perceptron: Summary

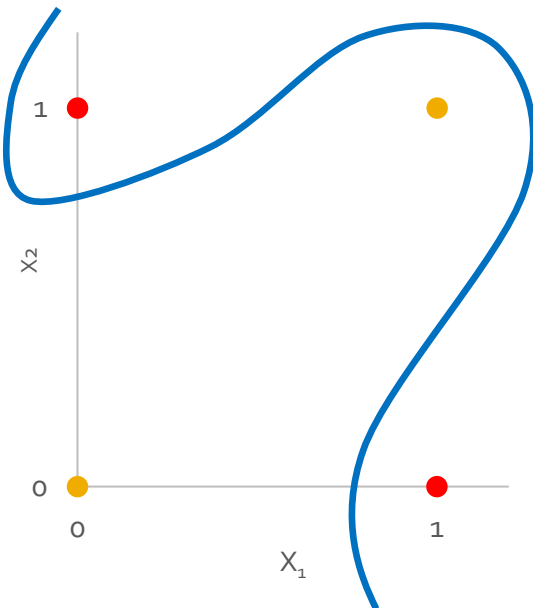
- **Online**, considers one instance at a time
- **Error-driven**: Only updates on misclassified examples
- Converges if the data are linearly separable
- Learned boundary is **linear**

Outline

- Neural Networks: Motivation and Biology
- The Perceptron
- Learning perceptron weights
- **Multilayer networks**

Limitations of the Perceptron

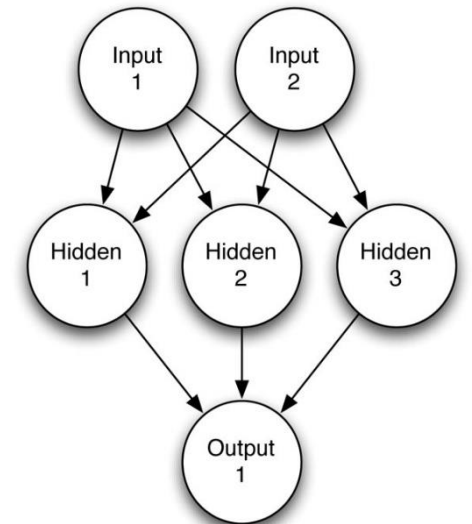
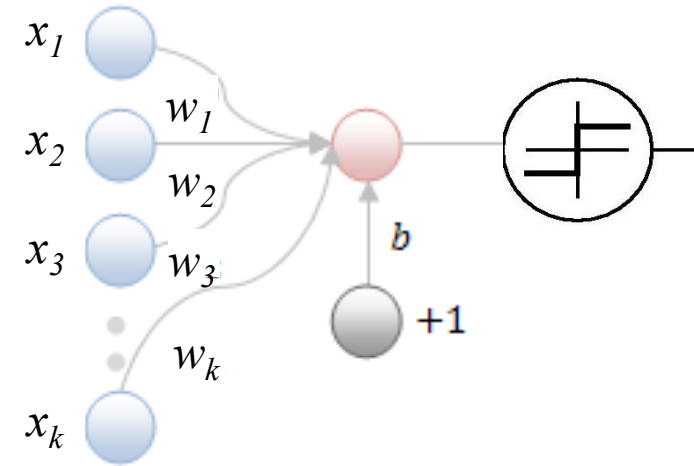
- Only works with linearly separable data
- These sort of problems led to “long winter” (1980’s)



x_1	x_2	y
1	1	-1
1	0	1
0	1	1
0	0	-1

Multilayer Networks

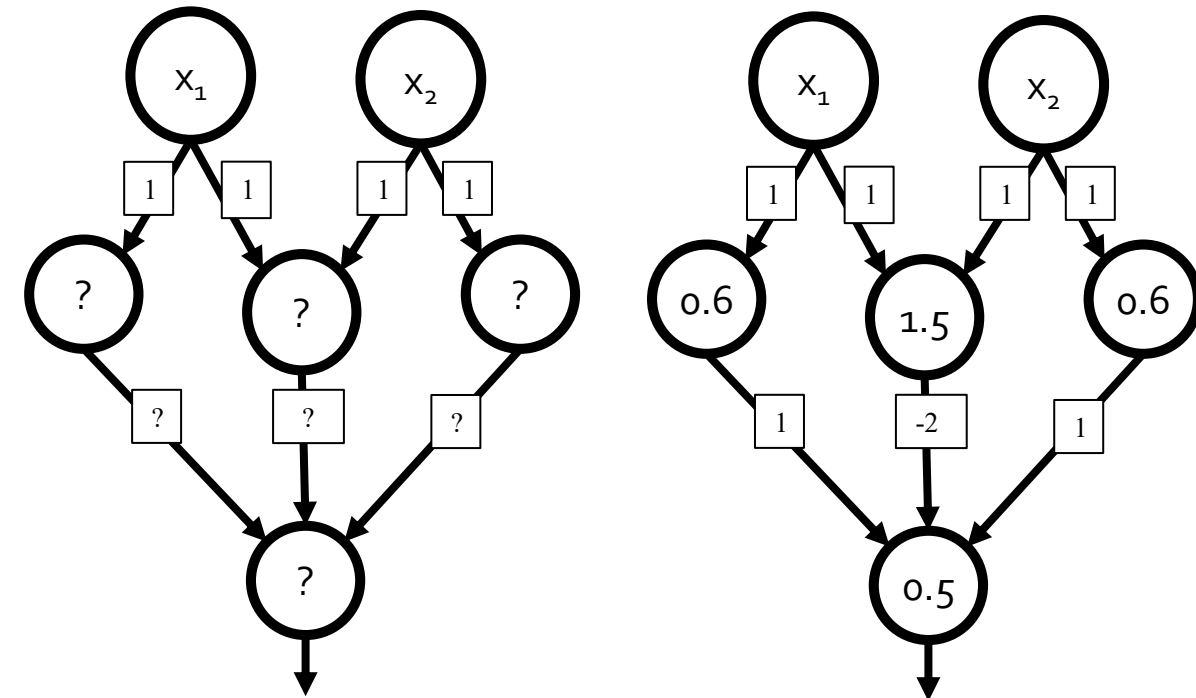
- Single-layer networks are limited
 - They can only learn hyperplanes
 - Most real-world problems are more complex
- What if we layer neurons?
 - **Multi-Layer Perceptron (MLP)**: an input layer, one or more hidden layers, and an output layer
 - Example two-layer network
 - Two layers of learnable weights: hidden+output layer
 - This allows for *very* powerful and complex computation!



XOR with an MLP

- We saw that XOR can't be solved with a single perceptron
- How about XOR with an MLP?
- Here's one example:

x1	x2	z
1	1	-1
1	0	1
0	1	1
0	0	-1



Your Turn: XOR

- Can you find weights that complete this XOR MLP?

